Fundementals of Computer Programming & OOP in Python

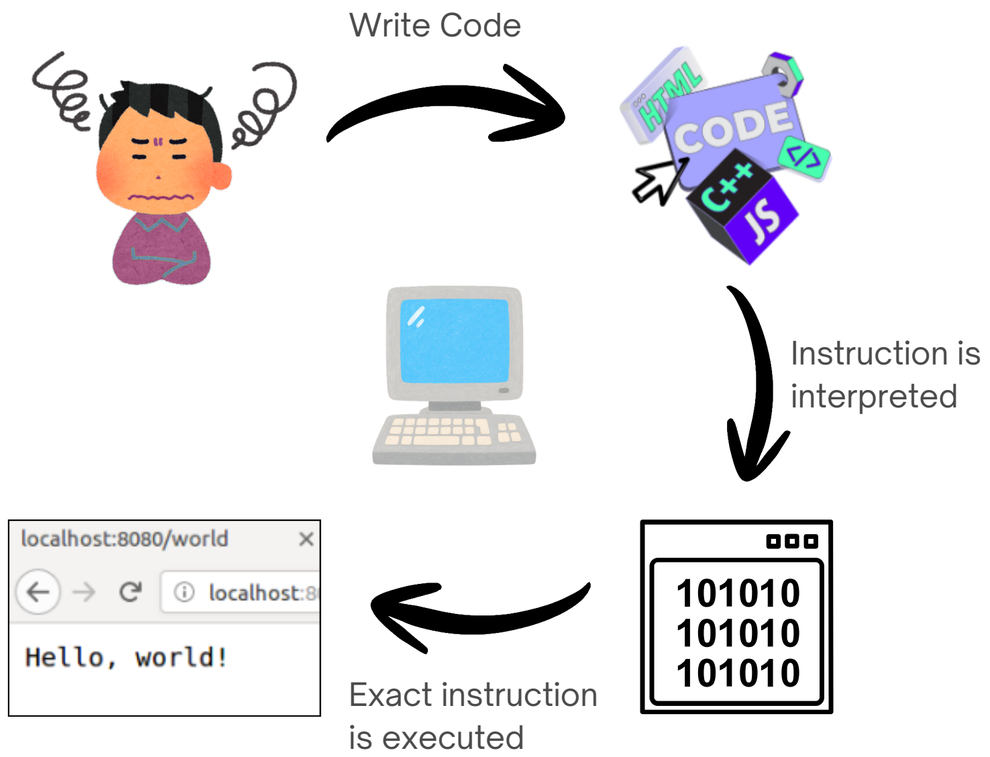
A Presentation by
Ali, Ayush, Dior, Shai,
Goku & Zachary

What is computer programming?

Computer Programming is writing precise instructions to a computer to solve a problem

Instructions are written in a programming language.

A Computer inteprets given code into binary and executes the exact instruction



What is Python?

Overview

Python is a **high-level programming language** designed to be easy to read and write. It is used by beginners and professionals alike for its **scalability**, branching into AI & Data analysis applications

History

In 1989, Python was created by Guido Van Rossum, at Centrum Wiskunde & Informatica (CWI), Netherlands

With its clear and user friendly syntax, it was built to create a language that is easy to read and write

It also helped handle exceptions easier and extended capabilities far beyond what most low level languages could do before



PyTorch

Machine Learning Framework

Open for Innovation
KNIME

Data Analysis Tool heightened by Python

Fundamental Concepts of Computer Programming

Demonstrated in Python

Variables

A Name That Stores Values

Name Value

```
class_venue = "Level 4.32"
```

Data Types

Different Types of Stored Values

String “”

```
iphoneduo = "£1,999"
```

Integer

```
count = 67
```

Boolean

```
is_student = True
```

Operators

A symbol that computes or compares

Addition
+

```
total = 5 + 2
```

Greater or Equal to
>=

```
score >= 50
```

Data Structures

Values Grouped Together by a **Variable**

```
list = [ 70,  80,  90 ]
print( list[ 0 ] )
#Output is 70
```

Fundamental Concepts of Computer Programming

Demonstrated in Python

Control Structures

These concepts steer the flow
3 Examples are Iteration, Selection & Sequence



Selection

Choosing a path

```
if score >= 50:
    print("Pass")
else:
    print("Fail")
```

Iteration

Repeating / Looping

```
for n in [1, 2, 3]:
    print(n)
#1
#2
#3
```

Sequence

Line 2 Overwrites Line 1

```
1  x = 1
2  x = 2
3  print(x)    #Output is 2
```

Fundamental Concepts of Computer Programming

Demonstrated in Python

Function

A named, reusable block
ie. doubling a number

defined

reused here

```
def double(x)
    return x * 2

print(double(4))    #8
```

Input/Output

Recieves data and outputs it

Input

Output

```
name = input("Name: ")
print("Hello ", name)
```

Algorithm

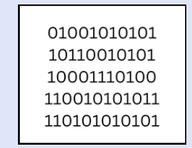
Planned steps to find a predictable solution

```
best = marks[0]
for m in marks:
    if m > best:
        best = m
print(best)
```

Programming Languages

Low Level Languages

Language written and structured for machines. Code is directly input into the CPU to be executed



Machine Language

Raw Binary Language, no translations needed



Assembly Language

Short Codes used for mapping CPU instructions, requires an assembler

High Level Langauges

Language written and structured for humans. It requires a compiler and interpreter to translate into binary for the CPU to run





Java

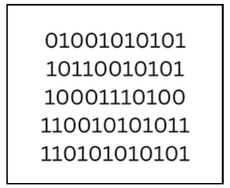


MySQL





Programming Languages

High Level Programming	VS	Low Level Programming
  Java  		 
Language is in English or English-like words		Language is in Binary or Short Codes
Easier for Humans to Interpret & Write		Harder for Humans to Read & Write
Slower to Process		Fast or Near Instant (Efficient)

Machine Language

The lowest level programming language, consisting of 0s and 1s. It can be executed on a cpu directly.

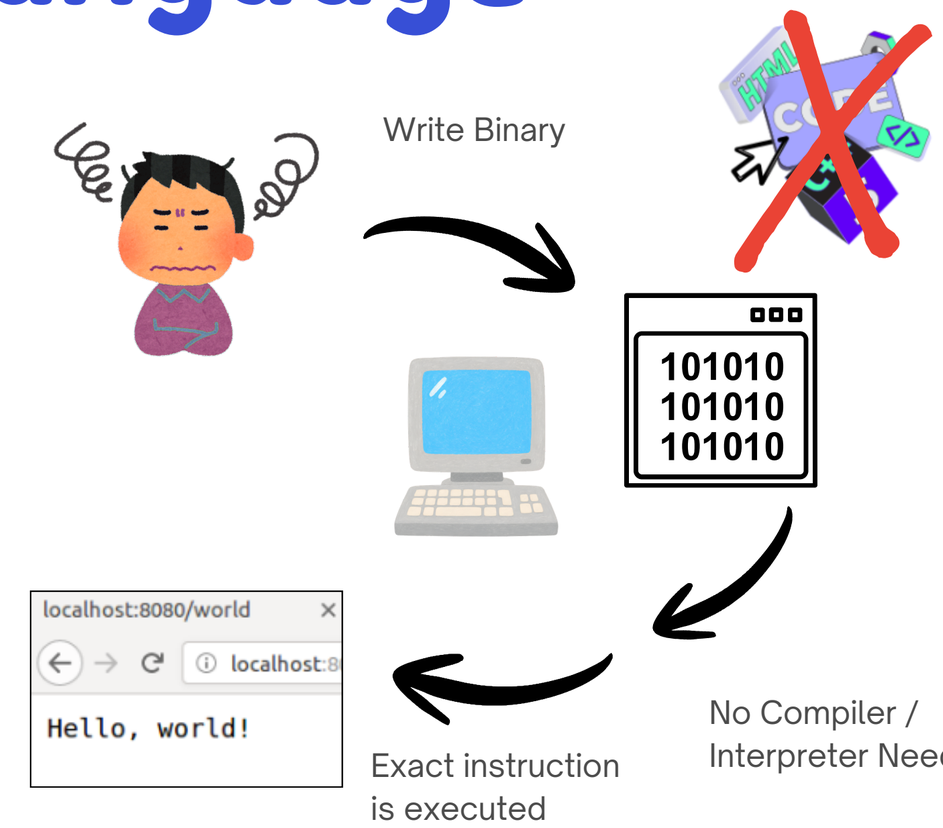
Key Characteristics



CPU/Architecture Dependency: Machine code is strictly tailored to a specific CPU instruction set architecture

Direct Execution: No compiler or interpreter is required at runtime.

Inflexible for Humans: Reading, writing, and debugging binary code by hand is nearly impossible for non-trivial applications



Assembly

One step above Machine Language. It replaces raw binary strings with readable human mnemonics. Does require an assembler to translate code into binary.

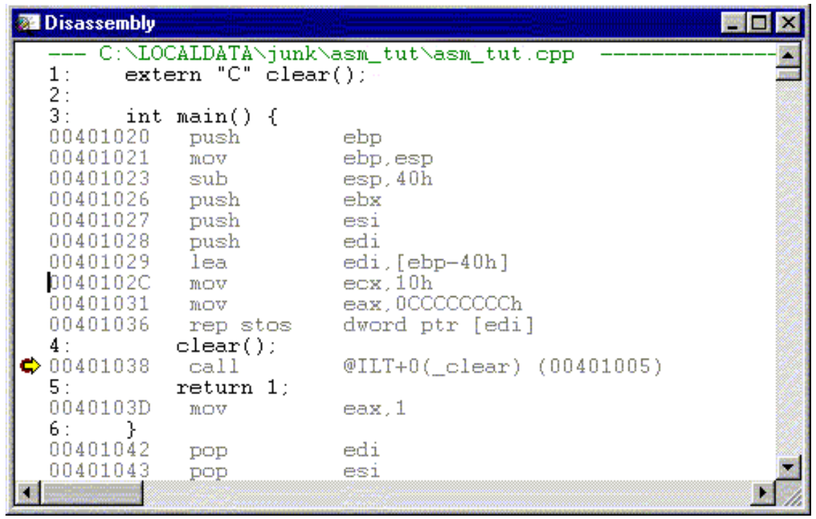
Key Characteristics

Assembly is not a compiler. Its assembler maps lines of mnemonics to one CPU instruction.

Also tied to one CPU / architecture

Assemblers do not abstract or optimize your logical structure. They blindly convert your explicit CPU instructions into binary bit patterns.

Assembler



Approaches to Programming

Object Oriented Programming

It is a method of writing programs as objects, objects that hold data and actions together. Each object has is a data field with its own attributes and behaviour.

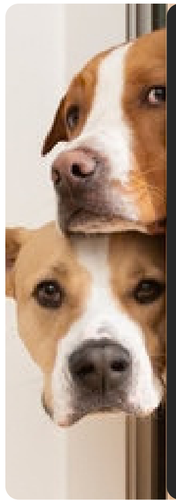
Data (Object)

Data fields are designed under its class. It refers to the attributes that would define the object.

Actions (Functions)

Actions are the functions that the data field can call, they would also be known as the object's attributes

Demonstrated in Python



```
class Dog:                                  #class
    def __init__(self, name):
        self.name = name                    #data

    def bark(self):                         #action
        print(self.name, "says woof")

rex = Dog("Rex")                            #object
rex.bark()                                  #Rex says woof
```

Object Oriented Programming

The Four Fundementals

Abstraction

Shows the most important information and hiding uneccessary details. It allows objects functions to be called without needing to know the full detail of the object

Inheritance

By building new base classes off other super classes, different objects can inherit behaviour or properties of another object and build on it with its own properties

Encapsulation

Keeping an object’s data and its code in one place. This remove the messy details and organises functions neatly

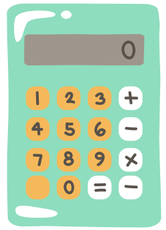
Polymorphism

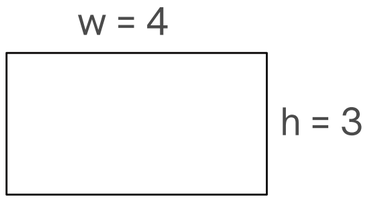
Allows the same object to exist in different forms, giving various outputs from the same function calls

Approaches to Programming

Procedural Programming

Writing a program as a sequence of steps, packed into functions. There is no object, a specific function is called which acts as an algorithm, solving specific problems with a specific output





Demonstrated in Python

```
def area(width, height):
    return width * height

        #procedure function for area of rectangle

w = 4
h = 3
print(area(w, h))                    #output is 12
```

Data (Object)

Data fields are designed under its class. It refers to the attributes that would define the object.

Actions (Functions)

Actions are the functions that the data field can call, they would also be known as the object's attributes

Approaches to Programming

Procedural Programming	VS	Object-Oriented Programming
Programs are a list of step-by-step instructions.		Programs are a collection of interacting objects.
Data and functions are generally separate		Data and the functions that operate on it are grouped together
Can be simpler for small programs		Can be more useful for large/complex programs

Syntax & Semantics

Syntax

Syntax is the grammar for coding. You must have correct syntax for your code to execute correctly.

Semantics

Semantics refers to the intended meaning of your code's function/output. Incorrect semantics leads to your code giving the wrong output despite being gramatically correct

If we ignore Syntax, our program will not run. If we ignore Semantics, our program will give the undesirable output

Demonstrated in Python



Syntax Error (Cannot Label Text without a String)

```
Dog = woof!    #missing paranthesis "" for a string
print(Dog);                                #output will fail
```

Semantic Error (String + Number)

```
Dog = "woof!"
print(Dog+1);  #cannot add a string and a number
```

Approaches to Interpretation



Compiled Programming VS Interpreted Programming



Compiled Programming	Interpreted Programming
A compiler translates the entire file into binary before it runs it as a whole	An interpreter reads and runs the program line by line
Errors will output as a whole, making debugging much easier & quicker	Error will appear when its line has been reached. Will have to rerun code after each fix
More efficient & faster once built	Slow, takes longer & more compute while running

Thank You

Any Questions?